SAVORIA RESTAURANT APP MVP

Software Requirements Specification (SRS) — Version 2

Assignment No. 2 — Restaurant App with Server and Database (React Native, Node.js, MongoDB)
Based on: SRS version 1 (Assignment No. 1, frontend only)
Semester: Fall 2026
Application: Savoria Restaurant App MVP
Technology: React Native + Expo Router, Node.js + Express, MongoDB (Mongoose)
Scope: Frontend, Express API server and MongoDB database

Revision history

Version	Description
1.0	Assignment 1. Frontend-only MVP using local mock data and AsyncStorage.
2.0	Assignment 2. Adds the Express server and MongoDB database. Sections 1.2, 1.3, 4.5, 5, 6.3, 6.4, 6.6, 8, 11 and 12 are updated or new. Old Sections 8 to 11 are now Sections 9 to 12.

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) version 2 defines the requirements for the Savoria Restaurant App MVP, updated from version 1 for Assignment 2. The application is a React Native app with a Node.js and Express backend and a MongoDB database, designed for restaurant customers and restaurant managers.

The purpose of Savoria is to allow customers to browse restaurant menu items, search and filter food, add items to a cart, apply promotional codes, reserve tables, place dine-in or takeaway orders, and track order progress.

The application also provides restaurant managers with a dashboard where they can view incoming orders, manage reservations, and manage menu items.

Version 1 of the MVP demonstrated these functions using React Native, Expo Router, React Hooks, Context API, reducers, custom hooks, local JavaScript mock data, and AsyncStorage, with no backend and no real database.

Version 2 keeps the same app and adds a Node.js and Express API server with a MongoDB database. All phones now send their data to the server, the server saves it in the database, and every phone reads from the same place. A change made by the manager on one phone is therefore visible to every customer.

1.2 Scope

In Scope

The Savoria Restaurant App MVP includes:

1. Customer login and signup through the server, with a token that is kept on the phone.
2. Customer signup with name, email, password, and confirmation password. The server always creates a customer account.
3. Manager login using a manager account stored in the database.
4. Menu browsing.
5. Menu categories including Starters, Mains, Desserts, and Drinks.
6. Daily Special identification.
7. Menu item availability display.
8. Menu search with debounce functionality.
9. Category filtering.

10. Menu sorting by price and name.
11. Favourite menu items.
12. Add to Cart functionality.
13. Cart quantity increment and decrement.
14. Remove cart items.
15. Special instructions for cart items.
16. Promotional code application.
17. Order summary calculation.
18. Service charge calculation.
19. Sales tax calculation.
20. Dine-in order selection.
21. Takeaway order selection.
22. Table reservation.
23. Table availability checking, done by the server.
24. Reservation validation, done by the app and by the server.
25. Reservation confirmation.
26. Reservation management by the manager (accept or decline).
27. Order creation on the server, with the total calculated from database prices.
28. Order tracking, refreshed every ten seconds from the server.
29. Manager order status updates (Pending, Preparing, Ready, Served) and cancellation while an order is Pending.
30. Customer profile.
31. Light and dark theme switching.
32. Manager Dashboard.
33. Manager order status management.
34. Manager reservation management.
35. Manager menu management (add, edit price, change availability, delete), saved on the server.
36. A Node.js and Express API server (Section 8).
37. A MongoDB database with the collections described in Section 5.
38. Password hashing and JSON Web Token (JWT) authentication.
39. Role-based access control: public, logged-in and manager-only endpoints.
40. Seed data for a manager, a customer, menu items and tables.
41. Responsive React Native interface.

Out of Scope

The following are NOT included:

- Real online payment processing.
- Credit/debit card processing.
- Real payment gateways.
- Push notifications.
- Email notifications.
- SMS notifications.
- Real-time push communication (WebSockets). The app polls the server instead.
- Delivery driver tracking.
- Real GPS tracking.
- External restaurant APIs.
- Firebase authentication.
- Production-level security infrastructure and public cloud hosting of the Express server. The server runs on the developer's laptop.

1.2.1 Problems Found in the A1 App and How Version 2 Solves Them

In A1 every phone had its own copy of the data. Using the A1 app showed these problems:

#	Problem in A1	Why it happened	Solution in version 2
1	The manager changes a price or marks a dish unavailable, but customers on other phones still see the old menu.	The menu lived in local mock data and local state on each phone.	The menu is stored in the MongoDB `menuitems` collection. Every app loads it with `GET /api/menu`. The manager edits it through protected endpoints, so every customer sees the change after a refresh.
2	An order placed on one phone is not visible on another phone or on the manager's phone, and it is lost when the app is reinstalled.	Orders were kept in OrdersContext and AsyncStorage on that phone only.	Orders are saved in the `orders` collection with `POST /api/orders`. Customers read their own orders with `GET /api/orders/my` and the manager reads all orders with `GET /api/orders`.
3	Two customers on two phones can book the same table at the same time.	Table availability was checked against each phone's own reservation list.	The server checks the table, the seating capacity, and whether the same table is already booked for that date and time. A clash returns 409.
4	An account created on one phone does not exist on another phone, and passwords were visible in the app's own data files.	Users were mock objects inside the app.	Users are stored in the `users` collection. Passwords are hashed with bcrypt. Login returns a JWT token.
5	The order status changed by a fake timer on each phone, so a manager's update never reached the customer.	Demo status progression ran locally.	The manager changes the status with `PATCH /api/orders/:id/status`. The customer's app polls `GET /api/orders/my` every ten seconds and shows the real status.
6	An item marked unavailable could still be ordered from a phone that had an old menu, and totals were calculated only on the phone.	The phone trusted its own copy of prices and availability.	The server loads the items from the database, rejects unavailable items, and calculates the total from database prices. It never trusts a total sent by the app.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification, a document describing system requirements.
MVP	Minimum Viable Product containing the core features needed for the prototype.
UML	Unified Modeling Language used to visually model software systems.
Hook	A React function that allows functional components to use state and other React features.
Context	React mechanism used to share data across components without passing props through every level.
Reducer	A pure function that calculates a new state from the current state and an action.
Mock Data	Local sample data used in version 1 instead of a real backend or database. It is replaced by the MongoDB database in version 2.
FlatList	React Native component used to efficiently render scrollable lists.
AsyncStorage	Local persistent key-value storage used by the application.
Expo Router	File-based navigation system used by the React Native application.
Component	A reusable UI or software building block in React Native.
Custom Hook	A reusable function beginning with `use` that contains reusable React logic.
API	Application Programming Interface. A set of server addresses the app can call to read or change data.
REST	A style of API design that uses HTTP methods (GET, POST, PUT, PATCH, DELETE) on resource addresses such as `/api/menu`.
Endpoint	One address of the API, made of an HTTP method and a path, for example `GET /api/menu`.
JSON	JavaScript Object Notation. A text format used to send data between the app and the server.
Node.js	A runtime that runs JavaScript on a computer, used to run the server.

Term	Definition
Express	A Node.js framework used to build the API server and define routes.
MongoDB	A document database that stores data as JSON-like documents.
Collection	A group of documents in MongoDB, similar to a table in a relational database.
Mongoose	A Node.js library that defines schemas and models and connects the server to MongoDB.
Middleware	A function that runs between the request and the route handler, for example to check a token.
JWT	JSON Web Token. A signed token the server gives after login. The app sends it with later requests to prove who the user is.
bcrypt	An algorithm used to hash passwords so the real password is never stored.
HTTP Status Code	A number in the server response showing the result, such as 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict and 500 Server Error.
CORS	Cross-Origin Resource Sharing. A setting that lets the app call the server from another address.
Environment variable	A setting kept in the `.env` file (such as `MONGO_URI`) and not committed to Git.
Seed script	A script (`npm run seed`) that fills the database with starting data.

---

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer	Browse menu, search food, manage cart, apply promo codes, reserve tables, place orders, view profile, change theme, and track orders.
Restaurant Manager	View incoming orders, update order statuses, manage reservations, and add/edit/toggle menu items.

2.2 User Stories

Customer User Stories

1. As a customer, I want to browse the restaurant menu, so that I can decide what to order.
2. As a customer, I want to see daily specials, so that I can quickly discover recommended food.
3. As a customer, I want to search menu items, so that I can find food without scrolling through the entire menu.
4. As a customer, I want to filter food by category, so that I can browse relevant dishes quickly.
5. As a customer, I want to check table availability, so that I can reserve a table before arriving.
6. As a customer, I want to book a table for a selected time, so that I can avoid waiting at the restaurant.
7. As a customer, I want to add food to a cart, so that I can prepare my order before checkout.
8. As a customer, I want to apply a promo code, so that I can receive a discount when a valid code is available.
9. As a customer, I want to choose dine-in or takeaway, so that I can select the option suitable for my visit.
10. As a customer, I want to track my order status, so that I know whether my food is pending, preparing, ready, or served.

Restaurant Manager User Stories

11. As a restaurant manager, I want to view incoming orders, so that I can manage customer orders.
12. As a restaurant manager, I want to update order statuses, so that customers can see the progress of their orders.
13. As a restaurant manager, I want to manage reservations, so that I can confirm or decline customer bookings.
14. As a restaurant manager, I want to add menu items, so that I can update the restaurant menu.
15. As a restaurant manager, I want to edit menu prices, so that displayed prices remain current.
16. As a restaurant manager, I want to toggle menu availability, so that unavailable food cannot be ordered.

---

3. Functional Requirements

3.1 Authentication

FR-01: The system shall allow a customer to log in using a valid email and password.

FR-02: The system shall authenticate users through the server (`POST /api/auth/login`) against the users stored in MongoDB.

FR-03: The system shall allow a new user to sign up using name, email, password, confirmation password, and role.

FR-04: The system shall validate the email format before authentication or signup.

FR-05: The system shall require a password of at least eight characters containing at least one digit.

FR-06: The system shall display an error when the password and confirmation password do not match.

FR-07: The system shall display a loading indicator while authentication is being simulated.

3.2 Menu

FR-08: The system shall display menu items loaded from the server (`GET /api/menu`).

FR-09: The system shall display menu items under Starters, Mains, Desserts, and Drinks categories.

FR-10: The system shall display a Daily Special badge for items marked as special.

FR-11: The system shall disable the Add to Cart button for unavailable items.

FR-12: The system shall visually distinguish unavailable menu items from available items.

FR-13: The system shall allow customers to select a menu category.

3.3 Search

FR-14: The system shall allow customers to search menu items by name.

FR-15: The system shall delay search execution using debounce functionality.

FR-16: The system shall display a friendly empty-state message when no menu item matches the search.

FR-17: The system shall allow customers to sort menu items by price or name.

3.4 Cart

FR-18: The system shall allow customers to add available menu items to the cart.

FR-19: The system shall allow customers to increase the quantity of a cart item.

FR-20: The system shall allow customers to decrease the quantity of a cart item.

FR-21: The system shall remove an item when its quantity reaches zero.

FR-22: The system shall allow customers to remove an item directly from the cart.

FR-23: The system shall allow customers to add special instructions to cart items.

FR-24: The system shall allow customers to apply valid promo codes.

FR-25: The system shall reject invalid promo codes with an error message.

FR-26: The system shall allow customers to remove an applied promo code.

3.5 Reservation

FR-27: The system shall allow customers to select a reservation date.

FR-28: The system shall provide hourly reservation slots from 12:00 to 22:00.

FR-29: The system shall disable a time slot when no suitable table is available.

FR-30: The system shall validate that the reservation date is not in the past.

FR-31: The system shall validate party size between 1 and 12 guests.

FR-32: The system shall validate Pakistani mobile numbers using the format `03XX-XXXXXXX`.

FR-33: The system shall require reservations to be at least one hour ahead of the current time.

FR-34: The system shall allow customers to select an available table.

FR-35: The system shall display a reservation confirmation before saving the booking.

FR-36: The system shall allow customers to cancel an existing reservation.

3.6 Orders

FR-37: The system shall allow customers to choose Dine-in or Takeaway.

FR-38: The system shall require a table selection for Dine-in orders.

FR-39: The system shall require a pickup time for Takeaway orders.

FR-40: The system shall create an order with an ID, items, total, type, status, and timestamp.

FR-41: The system shall create new orders with Pending status.

FR-42: The system shall calculate subtotal, service charge, tax, discount, and grand total.

FR-43: The system shall display an Order Summary before placing an order.

FR-44: The system shall display an Order Tracking screen after a successful order.

FR-45: The system shall let the manager update the order status from Pending to Preparing, Ready, and Served through the server. The customer's app shall refresh the status from the server every ten seconds.

3.7 Manager Dashboard

FR-46: The system shall display the Manager Dashboard only to users with the manager role.

FR-47: The system shall display incoming orders to the manager.

FR-48: The system shall allow the manager to update order status.

FR-49: The system shall display customer reservations to the manager.

FR-50: The system shall allow the manager to confirm or decline reservations.

FR-51: The system shall allow the manager to add menu items.

FR-52: The system shall allow the manager to edit menu item prices.

FR-53: The system shall allow the manager to toggle menu item availability.

FR-54: The system shall update customer menu data when a manager changes menu availability or pricing.

---

4. Non-Functional Requirements

4.1 Usability

The system shall provide a simple and understandable interface for customers and managers. Buttons, labels, status indicators, forms, and navigation controls shall be clearly visible and readable.

4.2 Performance

Menu lists shall use FlatList for efficient rendering and scrolling. Search shall use debounce to avoid unnecessary filtering during every keystroke. Expensive derived calculations such as order totals shall use useMemo where appropriate.

4.3 Responsiveness

The interface shall adapt to different mobile screen sizes and orientations supported by React Native. Layouts shall use flexible dimensions, spacing, and scrollable containers rather than fixed desktop-only dimensions.

4.4 Maintainability

The source code shall be organized into reusable folders including:

- `src/components`
- `src/screens`
- `src/context`
- `src/reducers`
- `src/hooks`
- `src/data`
- `src/navigation`

Reusable components and custom hooks shall prevent unnecessary duplication.

4.5 Data Handling

All shared data (users, menu items, tables, reservations and orders) shall be stored in MongoDB and accessed only through the Express API. The app shall keep only the login token and the user's name, email and role in AsyncStorage so that the user stays logged in. Logging out shall clear both.

The server shall hash passwords with bcrypt, never return a password or password hash, and keep secrets (`MONGO_URI`, `JWT_SECRET`) in a `.env` file that is not committed to Git.

The server shall validate every request (required fields, valid IDs, allowed values, price and quantity limits) and shall not depend on validation done in the app.

4.6 Reliability

The system shall provide loading, validation, error, Retry, pull-to-refresh and empty states where required. If the server cannot be reached, the app shall show a useful error message. Timers and intervals, including the ten-second order polling, shall be cleared during component cleanup.

4.7 Theme Consistency

All screens shall obtain colors from ThemeContext so that switching between light and dark mode updates the application consistently.

---

5. Database Design (MongoDB)

The mock data sets of version 1 are now collections in the MongoDB database `restaurant_app`. Every document also gets an `_id` (ObjectId) automatically. Collections that have timestamps also store `createdAt` and `updatedAt`.

Collection	Fields and Types	Relationships and Rules
Users (`users`)	`name` String; `email` String; `password` String (bcrypt hash); `role` String; `createdAt`, `updatedAt` Date	`name`, `email` and `password` are required. `email` is unique, lowercase and trimmed. `role` is `customer` or `manager` (default `customer`). Signup always creates a customer. The password is never returned by the API.
MenuItems (`menuitems`)	`name` String; `description` String; `category` String; `price` Number; `image` String; `isSpecial` Boolean; `available` Boolean; `createdAt`, `updatedAt` Date	`name`, `description`, `category` and `price` are required. `price` must be 0 or more. `image` defaults to an empty string, `isSpecial` to false and `available` to true. Menu items are referred to by Orders.
Tables (`tables`)	`tableNumber` Number; `seats` Number; `status` String; `createdAt`, `updatedAt` Date	`tableNumber` and `seats` are required, at least 1. `tableNumber` is unique. `status` is `available` or `reserved` (default `available`). Availability for a given date and time is decided from the Reservations collection.
Reservations (`reservations`)	`user` ObjectId; `table` ObjectId; `date` String (YYYY-MM-DD); `time` String (HH:mm); `guests` Number; `status` String; `createdAt`, `updatedAt` Date	`user` refers to Users and `table` refers to Tables. All fields except `status` are required. `guests` is at least 1 and must not be more than the table's `seats`. `status` is `Pending`, `Accepted` or `Declined` (default `Pending`). The same table cannot have two non-declined reservations for the same `date` and `time` (409).
Orders (`orders`)	`user` ObjectId; `items` array of {`menuItem` ObjectId, `name` String, `quantity` Number, `price` Number}; `totalAmount` Number; `status` String; `createdAt`, `updatedAt` Date	`user` refers to the Users document of the customer who placed the order, and each `menuItem` refers to MenuItems. The item `name` and `price` are copied into the order so later menu changes do not alter old orders. `quantity` is a whole number from 1 to 50. `totalAmount` is calculated by the server from database prices. `status` is `Pending`, `Preparing`, `Ready`, `Served` or `Cancelled` (default `Pending`).

5.1 Relationships

- One User has many Orders (`orders.user` → `users._id`).

- One User has many Reservations (`reservations.user` → `users._id`).
- One Table has many Reservations (`reservations.table` → `tables._id`).
- One Order contains many MenuItems through its `items` array (`orders.items.menuItem` → `menuitems._id`).

5.2 Validation Rules

- Required fields, unique fields, enums and minimum values are enforced by the Mongoose schemas.
- Invalid ObjectIds return 400 and never crash the server.
- Order status can only follow the order lifecycle in Figure 4. Cancellation is allowed only while the order is Pending.
- A reservation is rejected if the guests are more than the seats (400) or the table is already booked for that date and time (409).

5.3 Seed Data

`npm run seed` clears the collections and inserts one manager, one customer (passwords hashed), sixteen menu items in the four categories, and six tables.

---

6. UML Diagrams

6.1 Use Case Diagram

Figure 1 — Savoria Restaurant App Use Case Diagram

The Use Case Diagram shows the interaction between the Customer and Restaurant Manager actors and the major functions provided by the Savoria Restaurant App. Customer activities include browsing the menu, searching, managing the cart, making reservations, placing orders, and tracking orders. Manager activities include managing orders, reservations, and menu items.

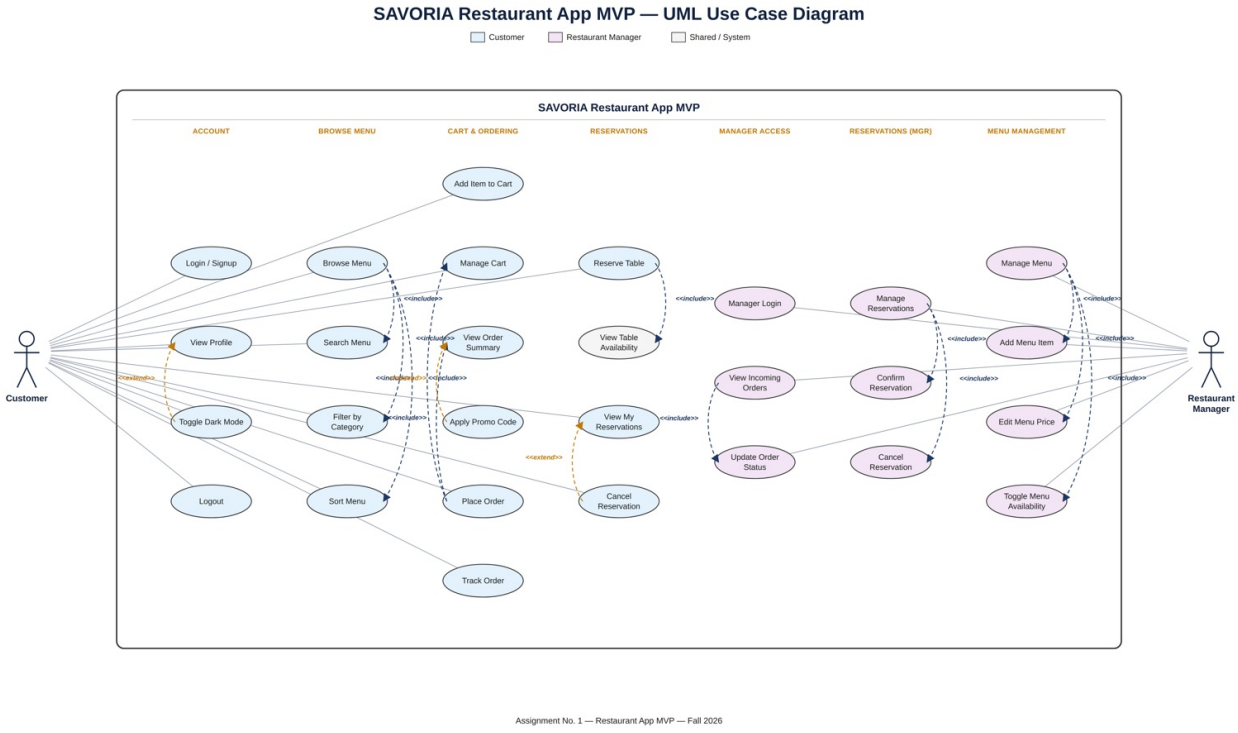


Diagram file: `A1/UML/Use_Case_Diagram.png`

6.2 Class Diagram

Figure 2 — Savoria Restaurant App Class Diagram

The Class Diagram models the main entities of the Restaurant App, including User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation, and Table. It shows attributes, operations, inheritance, relationships, and multiplicities between these classes.

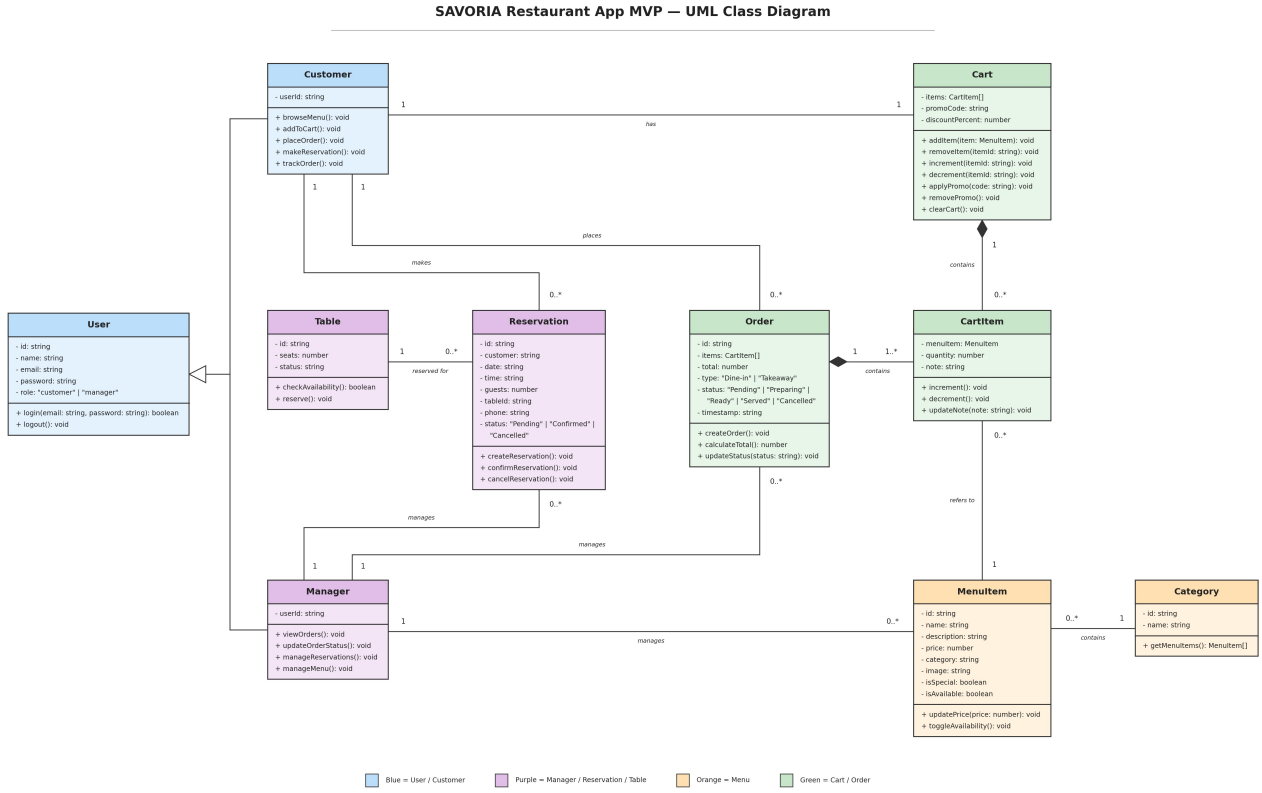
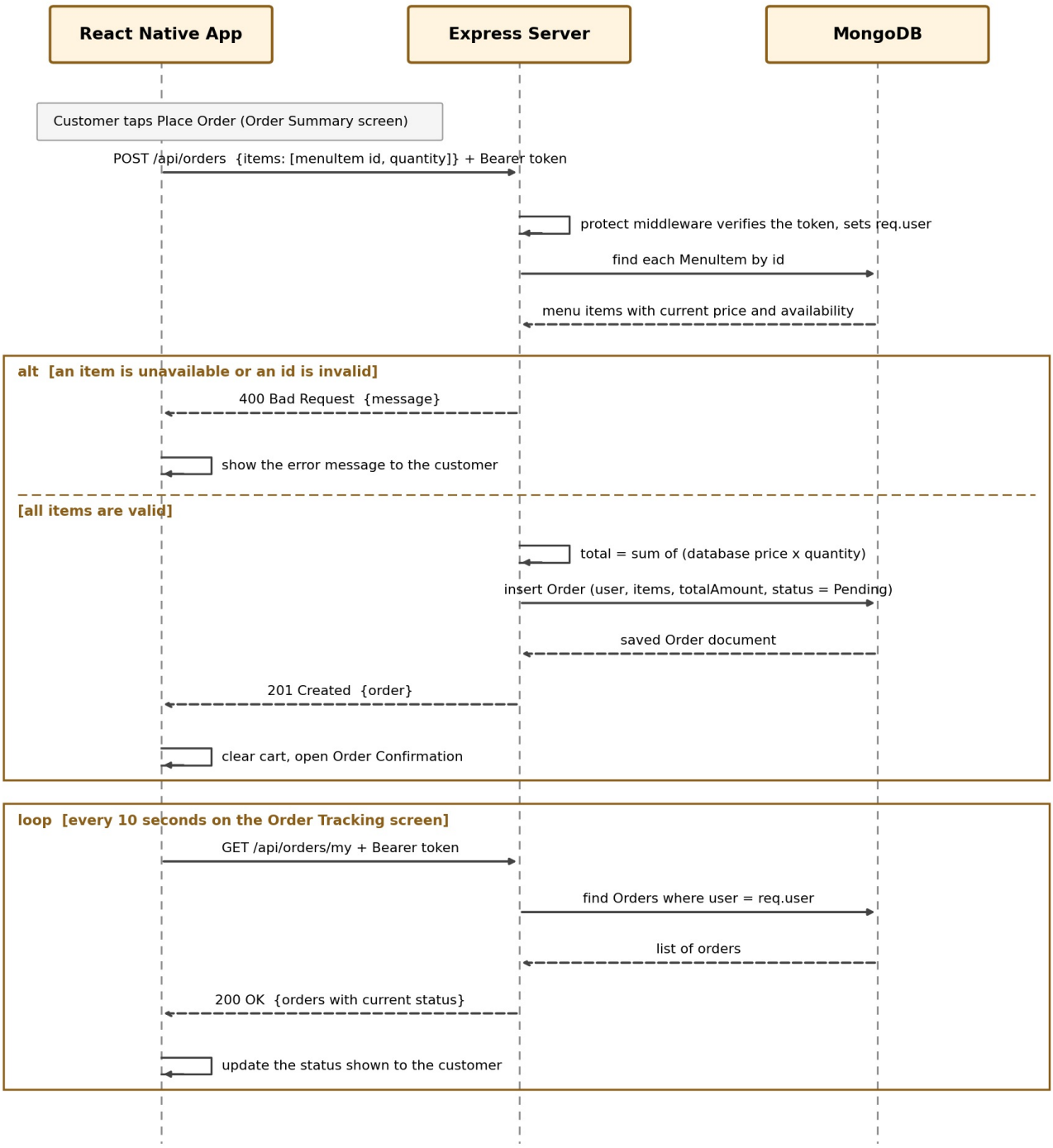


Diagram file: `A1/UML/Class_Diagram.png`

The classes still describe the main entities. In version 2 these entities are stored as the MongoDB collections in Section 5.

6.3 Sequence Diagram

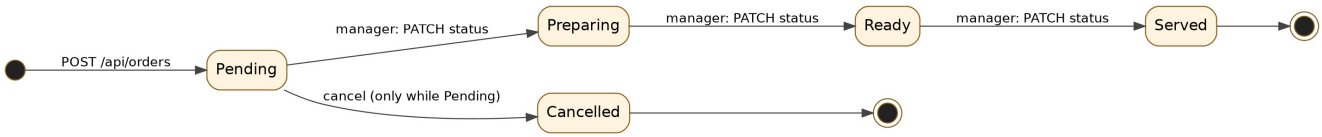
Figure 3 — Customer Places an Order Sequence Diagram (version 2)



The diagram was redrawn for version 2 with three lifelines: the React Native App, the Express Server and MongoDB. The app sends `POST /api/orders` with the token. The server checks the token, loads the menu items from MongoDB, rejects the order with 400 if an item is unavailable, and otherwise calculates the total from database prices, saves the order as Pending and returns 201. The Order Tracking screen then polls `GET /api/orders/my` every ten seconds. The version 1 diagram is kept in `A1/UML/Sequence_Diagram.png`.

6.4 State Machine Diagram

Figure 4 — Order Lifecycle State Machine Diagram (version 2)



The order begins in Pending status and can move to Preparing, Ready and Served. In version 2 the server allows cancellation only while the order is Pending, and any other change returns 400. Served and Cancelled are terminal states. The version 1 diagram (which also allowed cancelling from Preparing) is kept in `A1/UML/State_Machine_Diagram.png`.

6.5 Component Diagram

Figure 5 — Savoria Restaurant App Component Diagram

The Component Diagram shows the frontend architecture of the React Native application. It represents screens, Context Providers, reducers, custom hooks, reusable components, local mock data, and Expo Router navigation. The diagram demonstrates how these frontend components depend on each other.

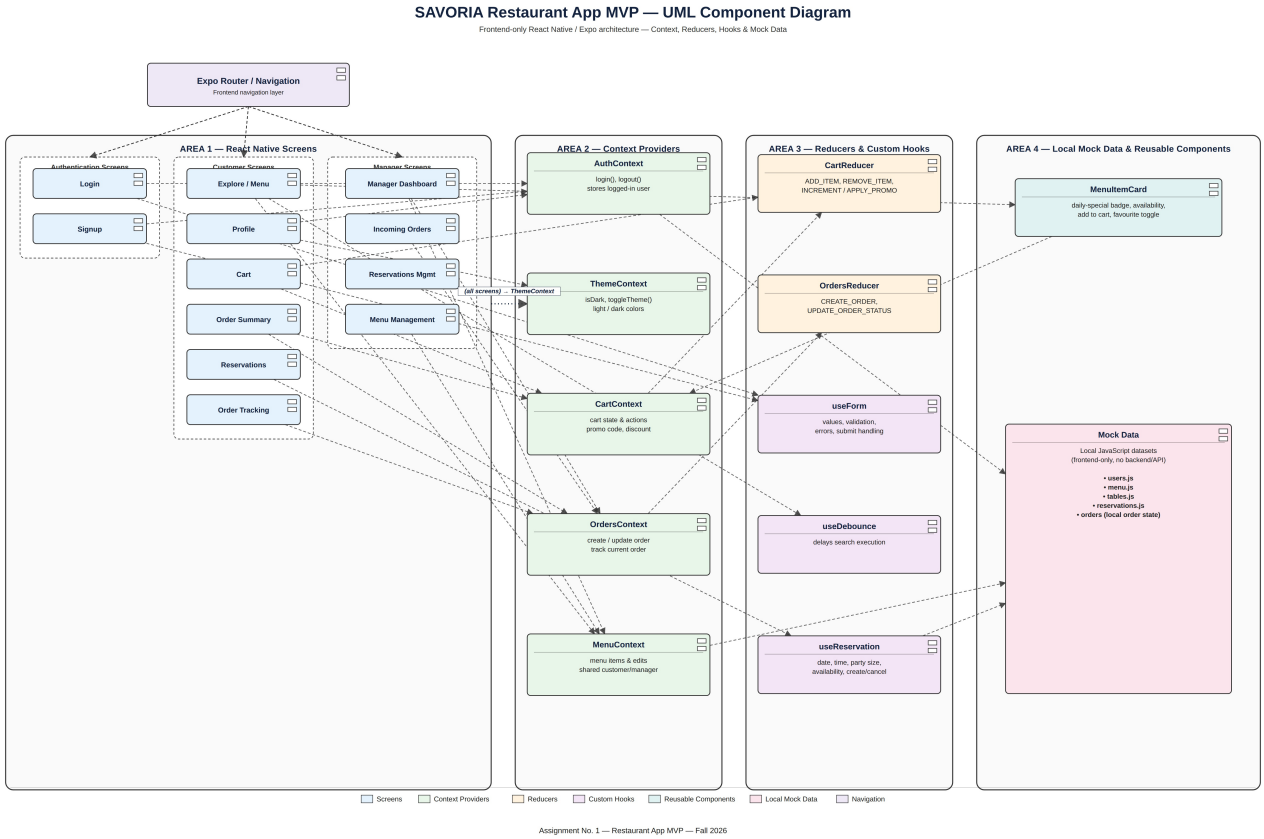
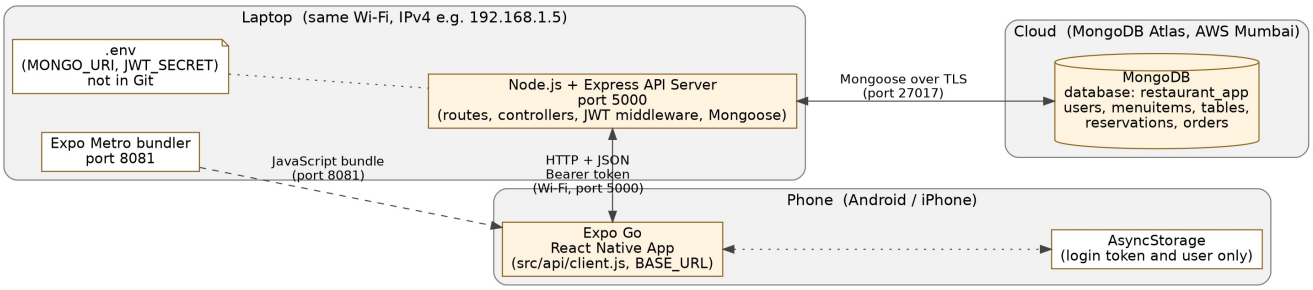


Diagram file: `A1/UML/Component_Diagram.png`

This diagram shows the frontend only. In version 2 the local mock data and the local storage of menu, orders and reservations are replaced by calls to the Express API, as shown in Figure 6.

6.6 Deployment Diagram

Figure 6 — Deployment Diagram



The phone runs the app in Expo Go. The laptop runs the Express server on port 5000 and the Expo Metro bundler on port 8081. The phone and the laptop must be on the same Wi-Fi, and the app calls the laptop's IP address (set in `BASE_URL` in `src/api/client.js`). The server connects to the MongoDB Atlas cluster in the cloud over TLS. The `.env` file with `MONGO_URI` and `JWT_SECRET` stays on the laptop and is not committed to Git.

---

7. MVP Frontend Development (React Native)

> **Note for version 2:** The *Local Data* subsections in this section describe how each screen worked in version 1. In version 2 the same screens keep their UI and hooks, but the data comes from the Express API (Section 8) and MongoDB (Section 5) instead of mock files and AsyncStorage. Login and signup use the auth endpoints, the menu comes from `GET /api/menu`, orders and reservations are saved and read through the order and reservation endpoints, and only the login token and user are stored locally.

7.1 Login and Signup Screen

Purpose

The Login and Signup screen provides local authentication for customers and restaurant managers.

UI Elements

- Email input
- Password input
- Full name input for signup
- Confirm password input
- Customer/Manager role selector
- Password visibility toggle
- Submit button
- ActivityIndicator
- Validation messages
- Login/Signup mode switch

Navigation Entry

The screen is entered when the application requires authentication.

Navigation Exit

A successful customer login navigates to the customer menu/explore screen. A successful manager login navigates to the Manager Dashboard.

Local Data

`src/data/users.js`

Hooks

useState: manages form values, errors, password visibility, selected role, and submitting state.

useContext: accesses AuthContext.

useForm: provides reusable form state and validation logic.

7.2 Menu Browsing Screen

Purpose

The Menu screen allows customers to browse restaurant food and drinks.

UI Elements

- Search bar
- Category chips
- Menu item cards
- Daily Special badge
- Price
- Description
- Availability indicator
- Add to Cart button
- Favourite button
- Sort controls
- Loading indicator
- Retry button
- Empty state
- Back-to-top button

Navigation Entry

Customer enters the screen after successful login or through the customer navigation.

Navigation Exit

Customer can navigate to Cart, Profile, Reservations, Order Summary, or Order Tracking.

Local Data

`src/data/menu.js`

Hooks

useState: selected category, search state, loading state, favourites, sorting, and UI state.

useEffect: loads mock menu data and handles refresh/lifecycle effects.

useRef: controls the search input, FlatList, debounce timer, previous search, and render counter.

useMemo: calculates filtered and sorted menu data.

useCallback: provides stable menu action handlers.

useDebounce: delays search execution.

7.3 Search and Scroll Controls

Purpose

This functionality allows customers to search menu items efficiently and navigate long menu lists.

UI Elements

- Search input
- Search icon
- Clear button
- Recent search suggestions
- Empty search result message
- FlatList
- Back-to-top floating button
- Debug render counter

Navigation Entry

Search functionality is available directly from the Menu screen.

Navigation Exit

Customers remain on the Menu screen while filtering results.

Local Data

Menu mock data.

Hooks

useRef: stores input references, FlatList reference, timeout values, previous query, and render count.

useState: stores search text, scroll position, and recent search terms.

useDebounce: performs delayed search processing.

7.4 Profile and Theme Screen

Purpose

The Profile screen displays authenticated user information and provides theme controls.

UI Elements

- User avatar
- Name
- Email
- Role
- Dark Mode switch
- Logout button

Navigation Entry

Customer or manager can open Profile from the application navigation.

Navigation Exit

The user can return to the previous screen or logout to the Login screen.

Local Data

Current user from AuthContext.

Hooks

useAuth: provides current user and logout functionality.

useTheme: provides current theme and theme switching.

7.5 Cart Screen

Purpose

The Cart screen allows customers to review and modify selected menu items before checkout.

UI Elements

- Cart item cards
- Quantity stepper
- Remove button
- Special instructions field
- Promo code input

- Discount display
- Item count
- Checkout button

Navigation Entry

Customer opens Cart after adding menu items.

Navigation Exit

Customer proceeds to Order Summary or returns to Menu.

Local Data

CartContext and local menu data.

Hooks

useReducer: manages complex cart state transitions.

useContext: accesses CartContext.

7.6 Order Summary Screen

Purpose

The Order Summary screen calculates and displays the final order cost before placement.

UI Elements

- Cart items
- Subtotal
- Service charge
- Sales tax
- Promo discount
- Grand total
- Dine-in/Takeaway selection
- Table or pickup-time selection
- Place Order button

Navigation Entry

Customer enters the screen from Cart.

Navigation Exit

Successful order placement navigates to Order Tracking.

Local Data

CartContext and OrdersContext.

Hooks

useMemo: calculates subtotal, service charge, tax, discount, and grand total.

useCallback: maintains stable event handlers where required.

useContext: accesses cart and order state.

7.7 Table Reservation Screen

Purpose

The Reservation screen allows customers to reserve available restaurant tables.

UI Elements

- Date selector
- Hourly time slots
- Guest count
- Table selection
- Contact phone
- Validation messages
- Confirmation modal
- My Reservations list
- Cancel button

Navigation Entry

Customer opens the Reservations screen from customer navigation.

Navigation Exit

Customer returns to the previous screen after booking or cancellation.

Local Data

`src/data/tables.js`

`src/data/reservations.js`

Hooks

useReservation: manages reservation business logic including availability, validation, creation, and cancellation.

useForm: manages reservation form values and validation.

useContext: accesses shared application state where required.

7.8 Order Tracking and Manager Dashboard

Order Tracking Purpose

The Order Tracking screen shows the current order lifecycle and progress.

UI Elements

- Order ID
- Order type
- Order total
- Current status
- Progress indicator
- Elapsed time
- Order timestamp

Order Tracking Navigation

The screen is entered after successful order placement.

Local Data

OrdersContext.

Hooks

useEffect: controls order progression timers and cleanup.

useState: manages elapsed time and screen state.

useContext: accesses OrdersContext.

Manager Dashboard Purpose

The Manager Dashboard provides restaurant managers with tools to manage incoming orders, reservations, and menu items.

UI Elements

Incoming Orders

- Order list
- Customer/order information
- Status controls
- Status update buttons

Reservations

- Reservation list
- Confirm button
- Decline/cancel button

Menu Management

- Add menu item
- Edit price
- Toggle availability

Navigation Entry

The Manager Dashboard is available only when the authenticated user's role is `manager`.

Navigation Exit

Manager can navigate between Incoming Orders, Reservations, and Menu Management.

Local Data

- OrdersContext
- MenuContext
- Reservation mock data
- Menu mock data

Hooks

useContext: accesses authentication, orders, menu, and theme state.

useReducer: manages order state.

useEffect: handles local persistence and loading.

useCallback: provides stable manager action handlers.

---

8. API Endpoints

The Express server exposes the following REST endpoints, all returning JSON. Protected endpoints need the header `Authorization: Bearer <token>`. The token is returned by the login (or register) endpoint and expires after one day.

Method	Endpoint	Access	Request Body	Response
GET	/api/health	Public	None	`status` and `time`
POST	/api/auth/register	Public	name, email, password	201 token and user (name, email, role). 409 if the email exists.
POST	/api/auth/login	Public	email, password	200 token and user (name, email, role). 401 if wrong.
GET	/api/menu	Public	None. Optional query: `category`, `search`	List of menu items
GET	/api/menu/:id	Public	None	One menu item. 400 invalid id, 404 not found.
POST	/api/menu	Manager	name, description, category, price, optional image, isSpecial, available	201 new menu item. 400 invalid data.
PUT	/api/menu/:id	Manager	Any of the menu item fields	200 updated menu item
DELETE	/api/menu/:id	Manager	None	200 message and the deleted item
POST	/api/orders	Logged in	items: list of {menuItem, quantity}	201 order with status Pending and server-calculated total. 400 if an item is unavailable or invalid.
GET	/api/orders/my	Logged in	None	The logged-in customer's own orders, newest first
GET	/api/orders	Manager	None	All orders with the customer's name and email
PATCH	/api/orders/:id/status	Logged in (customer: own order, cancel only)	status	200 updated order. 400 invalid transition. 403 if not allowed.
GET	/api/tables	Logged in	None	List of tables with seats and availability
POST	/api/reservations	Logged in	table, date, time, guests	201 reservation (Pending). 400 too many guests. 404 table not found. 409 table already booked.
GET	/api/reservations/my	Logged in	None	The logged-in customer's own reservations
GET	/api/reservations	Manager	None	All reservations with customer and table details
PATCH	/api/reservations/:id	Manager	status (Accepted or Declined)	200 updated reservation

8.1 Status Codes Used

Code	Meaning in this API
200 OK	The request worked.
201 Created	A new item, order, reservation or account was created.
400 Bad Request	Missing or invalid data, invalid ID, unavailable item, too many guests, or an invalid status change.
401 Unauthorized	No token, an invalid or expired token, or a wrong email or password.
403 Forbidden	The user is logged in but not allowed (for example a customer calling a manager-only endpoint).
404 Not Found	The item, order, reservation or table does not exist, or the route does not exist.
409 Conflict	The email is already registered, or the table is already booked for that date and time.
500 Server Error	An unexpected error on the server.

8.2 Business Rules Enforced by the Server

- The server calculates the order total from database prices and ignores any total sent by the app.
- The server rejects unavailable menu items.
- Order status moves only in the order Pending, Preparing, Ready, Served. A customer may cancel only their own order and only while it is Pending.
- Menu viewing is public. Creating, editing and deleting menu items needs a valid token and the manager role.

---

9. React Hooks Used in the MVP

Hook	Usage
useState	Form state, filters, theme UI, search, favourites, reservation values, and order type.
useEffect	Data loading, timers, persistence, cleanup, and lifecycle operations.
useRef	Search input, FlatList, timers, previous query, and render counter.
useContext	Authentication, theme, cart, menu, and order state.
useReducer	Cart and order state transitions.
useMemo	Filtered menu data and order total calculations.
useCallback	Stable event handlers for optimized components.
React.memo	Prevents unnecessary MenuItemCard re-renders.
Custom Hooks	Reusable form, debounce, and reservation business logic.

---

10. Frontend Data and State Management

The application uses React state management rather than a third-party state management library.

The major shared states are:

- Authentication state through AuthContext.
- Theme state through ThemeContext.
- Cart state through CartContext and cartReducer.
- Menu state through MenuContext.
- Orders state through OrdersContext and ordersReducer.

In version 2, shared data is loaded from and saved to the server through `src/api/client.js` and the `useApi` hook. MenuContext, AuthContext and the order and reservation screens call the API, and the database is the single source of truth.

AsyncStorage keeps only the login token and the logged-in user's details.

---

11. Client–Server Architecture

Savoria now uses a three-tier architecture:

```
React Native / Expo App  →  Express API Server  →  MongoDB Database
```

- **App:** React Native, Expo, Expo Router, Context API, reducers, hooks and custom hooks. The API client (`src/api/client.js`) keeps `BASE_URL` in one place, adds the JSON headers and the token, and turns server errors into readable messages. The `useApi` hook returns `data`, `loading`, `error` and `refetch`.

- **Server:** Node.js and Express with the folders `config`, `models`, `routes`, `controllers` and `middleware`, plus `server.js` and `seed.js`. Routes only map URLs to controller functions. The `protect` and `managerOnly` middleware handle authentication and the manager role.
- **Database:** MongoDB Atlas, accessed through Mongoose.

12. Conclusion

The Savoria Restaurant App MVP builds on Assignment No. 1 and Assignment No. 2 to provide the core customer and restaurant manager workflows.

Customers can authenticate, browse and search the menu, manage a cart, apply promotional codes, reserve tables, place orders, and track order progress. Restaurant managers can manage incoming orders, reservations, and menu availability.

Assignment No. 1 established the React Native frontend and its hooks-based architecture. Assignment No. 2 connects the app to an Express API server and MongoDB database. Menu changes, orders, reservations, and user accounts are shared across devices, so manager updates can be seen by customers and bookings and orders persist in the database.